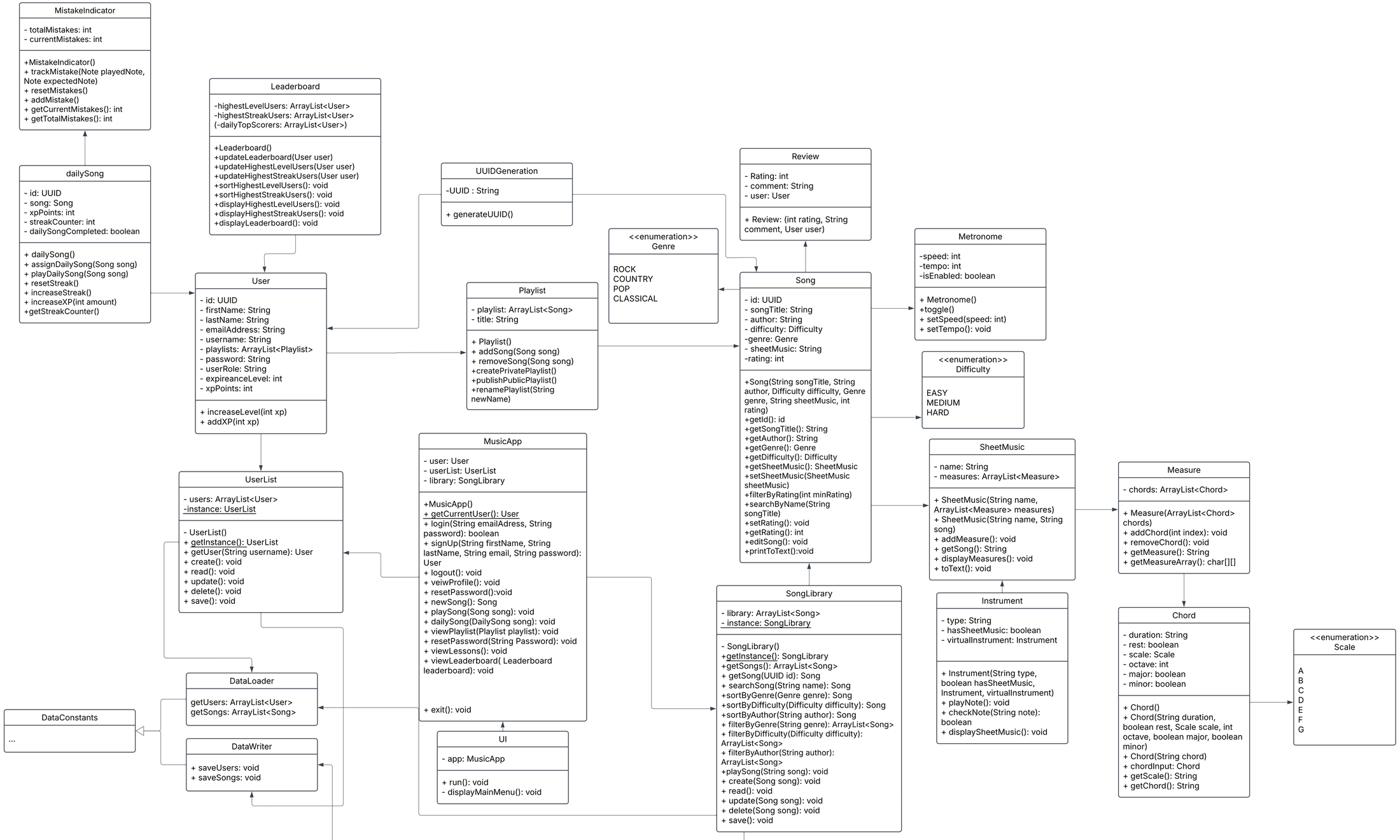

MistakeIndicator
- totalMistakes: int
- currentMistakes: int
+MistakeIndicator()
+ trackMistake(Note playedNote, Note expectedNote)
+ resetMistakes()
+ addMistake()
+ getCurrentMistakes(): int
+ getTotalMistakes(): int
Leaderboard
-highestLevelUsers: ArrayList<User>
-highestStreakUsers: ArrayList<User>
(-dailyTopScorers: ArrayList<User>)
+Leaderboard()
+updateLeaderboard(User user)
+updateHighestLevelUsers(User user)
+updateHighestStreakUsers(User user)
+sortHighestLevelUsers(): void
+sortHighestStreakUsers(): void
+displayHighestLevelUsers(): void
+displayHighestStreakUsers(): void
+displayLeaderboard(): void
dailySong
- id: UUID
- song: Song
- xpPoints: int
- streakCounter: int
- dailySongCompleted: boolean
+ dailySong()
+ assignDailySong(Song song)
+ playDailySong(Song song)
+ resetStreak()
+ increaseStreak()
+ increaseXP(int amount)
+getStreakCounter()
UUIDGeneration
-UUID : String
+ generateUUID()
Review
- Rating: int
- comment: String
- user: User
+ Review: (int rating, String comment, User user)
<<enumeration>>
Genre
ROCK
COUNTRY
POP
CLASSICAL
Metronome
-speed: int
-tempo: int
-isEnabled: boolean
+ Metronome()
+toggle()
+ setSpeed(speed: int)
+ setTempo(): void
User
- id: UUID
- firstName: String
- lastName: String
- emailAddress: String
- username: String
- playlists: ArrayList<Playlist>
- password: String
- userRole: String
- expireanceLevel: int
- xpPoints: int
+ increaseLevel(int xp)
+ addXP(int xp)
Playlist
- playlist: ArrayList<Song>
- title: String
+ Playlist()
+ addSong(Song song)
+ removeSong(Song song)
+createPrivatePlaylist()
+publishPublicPlaylist()
+renamePlaylist(String newName)
Song
- id: UUID
- songTitle: String
- author: String
- difficulty: Difficulty
-genre: Genre
- sheetMusic: String
-rating: int
+Song(String songTitle, String author, Difficulty difficulty, Genre genre, String sheetMusic, int rating)
+getId(): id
+getSongTitle(): String
+getAuthor(): String
+getGenre(): Genre
+getDifficulty(): Difficulty
+getSheetMusic(): SheetMusic
+setSheetMusic(SheetMusic sheetMusic)
+filterByRating(int minRating)
+searchByName(String songTitle)
+setRating(): void
+getRating(): int
+editSong(): void
+printToText():void
<<enumeration>>
Difficulty
EASY
MEDIUM
HARD
UserList
- users: ArrayList<User>
-instance: UserList
- UserList()
+ getInstance(): UserList
+ getUser(String username): User
+ create(): void
+ read(): void
+ update(): void
+ delete(): void
+ save(): void
MusicApp
- user: User
- userList: UserList
- library: SongLibrary
+MusicApp()
+ getCurrentUser(): User
+ login(String emailAdress, String password): boolean
+ signUp(String firstName, String lastName, String email, String password): User
+ logout(): void
+ veiwProfile(): void
+ resetPassword():void
+ newSong(): Song
+ playSong(Song song): void
+ dailySong(DailySong song): void
+ viewPlaylist(Playlist playlist): void
+ resetPassword(String Password): void
+ viewLessons(): void
+ viewLeaderboard( Leaderboard leaderboard): void
+ exit(): void
SheetMusic
- name: String
- measures: ArrayList<Measure>
+ SheetMusic(String name, ArrayList<Measure> measures)
+ SheetMusic(String name, String song)
+ addMeasure(): void
+ getSong(): String
+ displayMeasures(): void
+ toText(): void
Measure
- chords: ArrayList<Chord>
+ Measure(ArrayList<Chord> chords)
+ addChord(int index): void
+ removeChord(): void
+ getMeasure(): String
+ getMeasureArray(): char[][]
DataLoader
getUsers: ArrayList<User>
getSongs: ArrayList<Song>
DataConstants
...
DataWriter
+ saveUsers: void
+ saveSongs: void
UI
- app: MusicApp
+ run(): void
- displayMainMenu(): void
SongLibrary
- library: ArrayList<Song>
- instance: SongLibrary
- SongLibrary()
+getInstance(): SongLibrary
+getSongs(): ArrayList<Song>
+ getSong(UUID id): Song
+ searchSong(String name): Song
+sortByGenre(Genre genre): Song
+sortByDifficulty(Difficulty difficulty): Song
+sortByAuthor(String author): Song
+ filterByGenre(String genre): ArrayList<Song>
+ filterByDifficulty(Difficulty difficulty): ArrayList<Song>
+ filterByAuthor(String author): ArrayList<Song>
+playSong(String song): void
+ create(Song song): void
+ read(): void
+ update(Song song): void
+ delete(Song song): void
+ save(): void
Instrument
- type: String
- hasSheetMusic: boolean
- virtualInstrument: Instrument
+ Instrument(String type, boolean hasSheetMusic, Instrument, virtualInstrument)
+ playNote(): void
+ checkNote(String note): boolean
+ displaySheetMusic(): void
Chord
- duration: String
- rest: boolean
- scale: Scale
- octave: int
- major: boolean
- minor: boolean
+ Chord()
+ Chord(String duration, boolean rest, Scale scale, int octave, boolean major, boolean minor)
+ Chord(String chord)
+ chordInput: Chord
+ getScale(): String
+ getChord(): String
<<enumeration>>
Scale
A
B
C
D
E
F
G